



UNLP. Facultad de Informática.

Algoritmos y Estructuras de Datos 2006.

Trabajo Práctico 4

Tiempos de Ejecución - Complejidad de Algoritmos

Objetivos

- Comprender el concepto de función de tiempo de ejecución
- Comprender la noción de peor caso en un algoritmo
- Calcular el tiempo para un algoritmo
- Comprender el concepto de orden de un tiempo de ejecución
- Comparar distintos ordenes para determinar cual es cota de cual

Ejercicio 1.

Se analizan los algoritmos Alg1 y Alg2, que realizan la misma tarea. Se determina que tienen tiempos de ejecución para el peor caso $T1(n) = 10.000n$ y $T2(n) = n^2$ respectivamente.

¿Bajo qué circunstancias se elegiría uno u otro? **Fundamentar** su respuesta.

Ejercicio 2.

Determinar si las siguientes sentencias son verdaderas o falsas. **Justificar** la respuesta.

a) 3^n es de $O(2^n)$

b) $n/\log_2(n)$ es de $O(\log_2(n))$

c) $[\log_2(n)]^2$ es de $O(\log_2(n))$

d) Si $p(n)$ es un polinomio de grado k , entonces $p(n)$ es $O(n^k)$. *Pista: Si $p(n) = a_k n^k + \dots + a_1 n + a_0$, utilizar como constante a $M = |a_k| + \dots + |a_1| + |a_0|$*

e)

$$\begin{cases} 3n+17 & \text{si } n < 100 \\ 317 & \text{si } n \geq 100 \end{cases} \quad \text{Tiene orden lineal}$$

f) $n + \log_2(n)$ es de $O(n)$

g)

$$\begin{cases} n, & \text{si } n > 100. \\ n^2, & \text{si } n \leq 100 \end{cases} \quad \text{Tiene orden cuadrático}$$

h) $n^{1/2} + 10^{20}$ es de $O(n^{1/2})$

Ejercicio 3.

Dadas las siguientes funciones decir cuál de los siguientes ordenamientos, según sus velocidades de crecimiento es el correcto. **Justificar** su respuesta.

a) $T1(n) = n \cdot \log_2(n)$, $T2(n) = (1/3)^n$, $T3(n) = 2^n + n^2$, $T4(n) = (3/2)^n$

☐ $T1 < T2 < T3 < T4$

☐ $T1 < T2 < T4 < T3$

☐ $T2 < T1 < T4 < T3$

☐ $T2 < T4 < T1 < T3$

b) $T1(n) = \log_2(\log_2 n)$, $T2(n) = n^{1/2}$, $T3(n) = \log_2 n$, $T4(n) = n + \log_2(n)$

☐ $T1 < T2 < T3 < T4$

☐ $T1 < T3 < T4 < T2$

☐ $T3 < T1 < T4 < T2$

☐ $T1 < T3 < T2 < T4$

Ejercicio 4.

- a) Acceda a la documentación en línea de la API de Java, a través de la siguiente url:
<http://java.sun.com/j2se/1.5.0/docs/api/index.html>
- b) Analice la clase Math, ¿qué característica en común tiene todos sus métodos?

Ejercicio 5.

```
i) public static void uno (int n) {  
    int i, j, k ;  
    int [] [] a, b, c;  
  
    a = new int [n] [n];  
    b = new int [n] [n];  
    c = new int [n] [n];  
  
    for ( i:=1; i ≤ n-1; i++) {  
        for ( j:=i+1; j ≤ n; j++) {  
            for ( k:=1; k ≤ j; k++) {  
                c[i][j] = c[i][j] + a[i][j]*b[i][j];  
            }  
        }  
    }  
}
```

```
ii) public static void tres (int n)  
{  
    int i, j, k, sum;  
  
    sum = 0;  
  
    for ( i:=1; i ≤ n; i++) {  
        for ( j:=1; j ≤ i2 ; j++) {  
            for ( k:=1; k ≤ j; k++) {  
                sum = sum + 1;  
            }  
        }  
    }  
}
```

```
iv) public static void cuatro (int  
n) {  
  
    int i, j, k, s ;  
  
    s = 0;  
    j = 1;  
  
    for ( i:=1; i ≤ n; i++) {  
        if (i mod 2)= 1 {  
            j:= 2 * j;  
        }  
        else {  
            for( k:=1; k ≤ j; k++) {  
                s:=s+1;  
            }  
        }  
    }  
}
```

```
iii) public static void dos (int  
n) {  
  
    int i, j, x, y ;  
  
    x = 0;  
    y = 0;  
  
    for ( i:=1; i ≤ n-1; i++) {  
        if (n mod 2)= 1 {  
            for( j := i; j ≤ n; j++) {  
                x:= x + 1;  
            }  
            for( j := 1; j ≤ i; j++) {  
                y:= y + 1;  
            }  
        }  
    }  
}
```

Para cada uno de los algoritmos presentados:

a) Escriba en Java una clase llamada **Algoritmos** e implemente los métodos de clase.

b) **Determinar** cuál es el peor caso. **Fundamentar** su respuesta.

c) **Expresar** en función de n el tiempo de ejecución del peor caso

d) **Establecer** el orden de dicha función usando notación *big-Oh*.

En el caso de ser necesaria tenga presente que la $\sum i^4$ desde $i=1$ a n es $n(n+1)(6n^3+9n^2+n-1)/30$. Y la $\sum i^2$ desde $i=1$ a n es $n(n+1)(2n+1)/6$.

Nota: Cada una de las clases nuevas que implemente ubíquela en un paquete de nombre "estructurasdedatos" (Hasta ahora las creaba en el paquete por defecto).

Ejercicio 6.

a) Escriba en Java una clase llamada **Bucles** y escriba en diferentes métodos de clase el código que se presenta a continuación.

b) Determine para cada uno de los siguientes bucles su complejidad:

(i)

```
int c = 1;
while ( c < n ) {
    algo_de _ O(1)
    c = 2 * c;
}
```

(ii)

```
for ( int i:=1; i ≤ n; i++) {
    int c = i;
    while ( c > 0 ) {
        algo_de _ O(1)
        c = c / 2
    }
}
```

Ejercicio 7.

Sea A una matriz de números enteros de dimensión $N \times N$, donde los elementos de cada fila y de cada columna están ordenados en orden no - decreciente ($\forall i, j \ A[i,j] \leq A[i+1,j]$ y $A[i,j] \leq A[i,j+1]$).

- Escribir una clase llamada **Buscador** e **implementar** un método de clase que, dado un valor x, determine si existen $i, j \in \{1,2,3,...,n\}$ tal que $x = A[i,j]$.
- Determinar** el orden del método desarrollado.
- Re-implementar** el método del inciso a) en menor tiempo

Ejercicio 8.

Dado un vector de dimensión n, donde sus elementos son ceros o unos. **Implemente** en la clase Buscador, un método de clase que ordene el mismo con **complejidad lineal**.

Ejercicio 9.

Escriba la clase **PilaMin** como subclase de la clase Pila (implementada en una práctica anterior). Las operaciones deben tener complejidad **O(1)**

Especificación de los métodos de PilaMin

min // Retorna, sin eliminar, el elemento menor de la pila.

push: elem // Agrega elem a la pila (sobrescribe el de Pila)

pop // Elimina y devuelve el elemento en el tope de la pila (sobrescribe el de Pila)